GUIA TECNICA, MAPA DE ARCHIVOS Y MANUAL DE MODIFICACIONES - KA

```
================================================================================
Proyecto:              KAS Store - Tienda Especializada en Mezclilla y Jeans
Ubicacion Comercial:   La Masica, Departamento de Atlantida, Republica de Honduras
Estudio de Diseno:     Mora-Grafic's Studio (https://mora-grafics-studio.com/)
Version Documentada:   1.0.0 (Fase 6 - Auditoria y Manual de Arquitectura)
Fecha de Elaboracion:  Octubre de 2026
Entorno de Ejecucion:  Node.js 22+ / React 19 / TypeScript / Vite / Tailwind CSS v4 /
================================================================================
```

INDICE GENERAL

1. [Objetivo y Alcance del Documento](#1-objetivo-y-alcance-del-documento)
2. [Auditoria Integral de la Estructura de Archivos](#2-auditoria-integral-de-la-estructura-de-archivos)
3. [Mapa de Archivos por Seccion de la Tienda](#3-mapa-de-archivos-por-seccion-de-la-tienda)
* [A. Identidad Visual y Diseno de Marca](#a-identidad-visual-y-diseno-de-marca)
* [B. Pagina de Inicio (Home)](#b-pagina-de-inicio-home)
* [C. Catalogo y Gestion de Productos](#c-catalogo-y-gestion-de-productos)
* [D. Pagina de Detalle de Producto](#d-pagina-de-detalle-de-producto)
* [E. Bolsa de Compras y Carrito](#e-bolsa-de-compras-y-carrito)
* [F. Checkout y Creacion de Pedidos](#f-checkout-y-creacion-de-pedidos)
* [G. Integracion de Pasarelas de Pago](#g-integracion-de-pasarelas-de-pago)
* [H. Panel Administrativo de la Tienda](#h-panel-administrativo-de-la-tienda)
* [I. Sistema Logistico y Envios en Honduras](#i-sistema-logistico-y-envios-en-honduras)
* [J. Canales de Contacto y WhatsApp](#j-canales-de-contacto-y-whatsapp)
* [K. SEO, Metadatos y Paginas Legales](#k-seo-metadatos-y-paginas-legales)
* [L. Notificaciones y Correos Electronicos](#l-notificaciones-y-correos-electronicos)
4. [Tabla Maestra de Modificaciones](#4-tabla-maestra-de-modificaciones)
5. [Mapas de Flujo de Datos (Diagramas Mermaid)](#5-mapas-de-flujo-de-datos)
6. [Inventario de Variables de Entorno](#6-inventario-de-variables-de-entorno)
7. [Base de Datos Relacional y Modelado de Datos](#7-base-de-datos-relacional-y-modelado-de-datos)
8. [Protocolo de Seguridad para Modificar el Codigo sin Romperlo](#8-protocolo-de-seguridad-para-modificar-el-codigo-sin-romperlo)
9. [Glosario y Guia Practica para el Propietario](#9-glosario-y-guia-practica-para-el-propietario)
10. [Estado de Verificacion, Limitaciones Conocidas y Observaciones](#10-estado-de-verificacion-limitaciones-conocidas-y-observacione

---

1. OBJETIVO Y ALCANCE DEL DOCUMENTO

Este manual ha sido elaborado especificamente para el propietario y los desarrolladores de **KAS Store**. Su proposito es servir como documento de consulta permanente para comprender la arquitectura real de la aplicacion, localizar de forma inmediata que archivo modificar ante cualquier cambio comercial y garantizar que el sistema opere con estabilidad, seguridad transaccional y cumplimiento legal en Honduras.

Reglas Clave de Mantenimiento

* **Fuente de verdad hibrida y tolerante a fallos:** El catalogo se alimenta de la base de datos relacional PostgreSQL a traves de '/api/products'. Si la base de datos no estuviera disponible, el sistema cuenta con un respaldo en memoria ('src/data/mockProducts.ts') para garantizar que la tienda nunca muestre una pantalla en blanco a los clientes.
* **Seguridad en pagos y PCI-DSS:** El sistema nunca almacena numeros de tarjeta ni CVV. Solo guarda identificadores de transaccion provistos por PixelPay o PayPal.
* **Inventario transaccional:** Toda orden bloquea las existencias en PostgreSQL con

'SELECT FOR UPDATE' para impedir la sobreventa de jeans.

---

2. AUDITORIA INTEGRAL DE LA ESTRUCTURA DE ARCHIVOS

2.1 Arbol Real del Proyecto

```
/
    index.html                          # Punto de entrada HTML con SEO y Schema.org
    server.ts                           # Servidor Node.js + Express (API REST y est
    package.json                        # Definicion de dependencias y scripts de ej
    tsconfig.json                       # Configuracion del compilador TypeScript
    vite.config.ts                      # Configuracion de compilacion de Vite y Tai
    metadata.json                       # Metadatos del entorno AI Studio
    firebase-applet-config.json         # Configuracion de proyecto Firebase
    CHECKOUT_Y_PAGOS.md                 # Bitacora de pasarelas de pago y sandbox
    DESPLIEGUE_Y_OPERACIONES.md         # Manual de infraestructura, Cloud Run y bac

    public/                             # Archivos publicos servidos directamente
        robots.txt                      # Directivas para rastreadores de motores de
        sitemap.xml                     # Mapa del sitio XML con rutas y productos
        uploads/                        # Directorio de imagenes subidas desde el pa
        assets/
            aistudio/
                Logotipo_KAS_Store_9c74b5ef.png  # Logotipo oficial de KAS Store

    src/                                # Codigo fuente del Frontend y Logica Compar
        main.tsx                        # Punto de entrada de React 19 (montaje en #
        App.tsx                         # Enrutador principal de vistas y layout glo
        index.css                       # Estilos globales y paleta @theme de Tailwi
        types.ts                        # Definicion de tipos TypeScript e interface

        assets/                         # Activos de diseno
            images/                     # Fotografias de modelos y tonalidades de me

        components/                     # Componentes reutilizables de UI
            Header.tsx                  # Barra de anuncio, navegacion, busqueda y b
            Footer.tsx                  # Pie de pagina, enlaces de servicio y copyr
            ProductCard.tsx             # Tarjeta de producto en catalogo y destacad
            ProductImage.tsx            # Renderizador seguro de imagenes con fallba
            CartDrawer.tsx              # Drawer deslizable lateral de la bolsa de c
            AccountModal.tsx            # Modal de cuenta cliente y panel de control
            SizeGuideModal.tsx          # Modal de guia de tallas (Hombre, Mujer, Ni
            WhatsAppButton.tsx          # Boton flotante interactivo de atencion por

        context/                        # Proveedores de estado global (React Contex
            AuthContext.tsx             # Autenticacion con Firebase y sincronizacio
            CartContext.tsx             # Estado de la bolsa, cantidades y totales e
            WishlistContext.tsx         # Gestion de favoritos sincronizada con back

        data/                           # Datos estaticos y diccionarios
            mockProducts.ts             # Catalogo base de jeans, imagenes hero y ca
            hondurasLocations.ts        # Lista de 18 departamentos y municipios de

        db/                             # Base de datos relacional PostgreSQL con Dr
            index.ts                    # Conexion al pool pg (soporta Cloud SQL y l
            schema.ts                   # Esquema relacional de tablas, tipos y rela
            seed.ts                     # Script de inicializacion con productos y c
            users.ts                    # Funciones de consulta y creacion de usuari
            drizzle.config.ts           # Configuracion para migraciones de Drizzle
```

```
        lib/                            # Bibliotecas auxiliares y utilidades
            cardValidation.ts           # Validacion Luhn y deteccion de marcas de t
            shippingRates.ts            # Tarifario y calculo oficial por zonas de H
            firebase.ts                 # Inicializacion del SDK cliente de Firebase
            firebase-config.ts          # Variables de configuracion de Firebase cli
            firebase-admin.ts           # Inicializacion de Firebase Admin para back

        middleware/                     # Interceptores de peticiones Express
            auth.ts                     # requireAuth, requireAdmin y optionalAuth

        pages/                          # Paginas o vistas principales de la aplicac
            HomePage.tsx                # Portada editorial, banner hero, destacados
            CatalogPage.tsx             # Catalogo con filtros de mezclilla, busqued
            ProductDetailPage.tsx       # Detalle del jean, selector de variantes y
            CartPage.tsx                # Vista completa de la bolsa con desglose de
            CheckoutPage.tsx            # Proceso de pago, direccion en Honduras y p
            FavoritesPage.tsx           # Lista de jeans guardados por el usuario
            PoliciesPage.tsx            # Centro de politicas legales, envios y devo
            NotFoundPage.tsx            # Pagina de error 404 personalizada

        services/                       # Capa de comunicacion y logica de negocio
            api.ts                      # Cliente HTTP frontend para interactuar con
            cartStorage.ts              # Persistencia de carrito y favoritos en loc
            paymentService.ts           # Logica transaccional de pagos, PixelPay y
```

---

3. MAPA DE ARCHIVOS POR SECCION DE LA TIENDA

A. Identidad Visual y Diseno de Marca

```
Elemento de Diseno | Archivo Responsable | Donde y Como Modificarlo
--- | --- | ---
**Logotipo oficial** | ‘/public/assets/aistudio/Logotipo_KAS_Store_9c74b5ef.png‘ | Re
**Paleta de Colores (Arena y Terracota)** | ‘/src/index.css‘ | En el bloque ‘@theme‘
**Tipografias** | ‘/index.html‘ e ‘/src/index.css‘ | Google Fonts en ‘index.html‘ (L.
**Estilos de Botones y Sombras** | Clases utilitarias de Tailwind en componentes | Lo
**Iconografia** | Todo el proyecto | Utiliza la libreria ‘lucide-react‘. Para cambiar
```

B. Pagina de Inicio (Home)

Archivo principal: ‘/src/pages/HomePage.tsx‘

* **Banner Principal / Hero:** Lineas 72 a 155. Contiene el logotipo en tamano destacado, el titulo *"Tu estilo, tu esencia"*, la descripcion del negocio y los botones *"Comprar coleccion"* y *"Descubrir novedades"*.
* **Imagen del Hero:** Constante ‘HERO_IMAGE‘ importada desde ‘/src/data/mockProducts.ts‘ (L. 3). Para cambiar la foto de portada, actualiza la ruta del archivo JPG en ‘mockProducts.ts‘.
* **Productos Destacados y Novedades:** Lineas 39-40. Se filtran automaticamente con ‘productsList.filter(p => p.isFeatured)‘ y ‘productsList.filter(p => p.isNew)‘.
* **Beneficios de Compra:** Lineas 406 a 460. Contiene los 4 pilares: *Envios a Todo Honduras*, *Mezclilla Autentica*, *Asesoria WhatsApp* y *Cambios Sencillos*.
* **Preguntas Frecuentes (FAQ):** Lineas 46 a 67. Arreglo ‘faqs‘ con preguntas y respuestas editables directamente en el codigo.

C. Catalogo y Gestion de Productos

Archivos involucrados:

* Frontend: ‘/src/pages/CatalogPage.tsx‘
* Backend API: ‘/server.ts‘ (Endpoint ‘GET /api/products‘, L. 78-202)
* Modelo de Datos: ‘/src/db/schema.ts‘ (‘products‘, ‘product_variants‘, ‘inventory‘, ‘product_images‘)
* Respaldo en memoria: ‘/src/data/mockProducts.ts‘

Como funciona la fuente de verdad?
1. Al cargar el catalogo, ‘CatalogPage.tsx‘ ejecuta ‘api.getProducts()‘.
2. El servidor consulta la tabla ‘products‘ en PostgreSQL, uniendo las imagenes (‘product_images‘) y las variantes con su stock real (‘inve
3. Si la base de datos responde correctamente, el estado ‘isDbSynced‘ pasa a ‘true‘ y muestra los productos de PostgreSQL.
4. Si la base de datos estuviera desconectada, la funcion atrapa el error y utiliza ‘MOCK_PRODUCTS‘ de forma transparente.

D. Pagina de Detalle de Producto

Archivo de Presentacion: ‘/src/pages/ProductDetailPage.tsx‘
Endpoint de Logica: ‘/server.ts‘ (‘GET /api/products/:slugOrId‘, L. 205-303)

* **Galeria de Imagenes:** Lineas 55-60 y componente ‘ProductImage.tsx‘. Permite cambiar la foto activa mediante miniaturas.
* **Selector de Tallas y Colores:** Lineas 160-230. Valida si la combinacion especifica tiene unidades en stock llamando a ‘isVariantOutOfStock()‘. Si esta agotada, el boton muestra *"Agotado en esta combinacion"*.
* **Acciones:**
* *Agregar a la Bolsa:* Ejecuta ‘addItem(product, selectedSize, selectedColor, quantity)‘ de ‘CartContext.tsx‘ y despliega el ‘CartDrawer‘.
* *Comprar Ahora:* Agrega el jean a la bolsa y redirige automaticamente a la vista de ‘checkout‘.

E. Bolsa de Compras y Carrito

Archivos involucrados:
* Estado Global: ‘/src/context/CartContext.tsx‘
* Persistencia Local: ‘/src/services/cartStorage.ts‘
* Vista Drawer Deslizable: ‘/src/components/CartDrawer.tsx‘
* Pagina Completa de Bolsa: ‘/src/pages/CartPage.tsx‘
* Validacion en Servidor: ‘/server.ts‘ (‘POST /api/checkout/calculate‘, L. 366-460)

Regla de Seguridad: El calculo visual del carrito en el cliente (‘subtotalHnl‘ y ‘shippingHnl‘ en ‘CartContext.tsx‘) es solo informativo. Cuando el usuario procede a pagar, el servidor en ‘/api/checkout/calculate‘ y en ‘/api/orders‘ recalcula de manera obligatoria los precios consultando los valores oficiales en la base de datos para impedir manipulaciones en el navegador.

F. Checkout y Creacion de Pedidos

Archivo Frontend: ‘/src/pages/CheckoutPage.tsx‘
Endpoints Backend: ‘/server.ts‘
* ‘POST /api/checkout/calculate‘ (L. 366): Calculo autoritativo de subtotal, descuento de cupon y tarifa de envio.
* ‘POST /api/orders‘ (L. 463-734): Creacion transaccional del pedido con reserva de inventario.
* ‘GET /api/orders/:orderNumber‘ (L. 937): Consulta de pedido por numero unico.

Flujo de Reserva de Inventario:
1. El backend abre una transaccion PostgreSQL con ‘client.query(’BEGIN’)‘.
2. Ejecuta ‘reserveInventory()‘ en ‘/src/services/paymentService.ts‘, aplicando ‘SELECT stock_quantity, reserved_quantity FROM invento
3. Si hay suficiente stock, incrementa ‘reserved_quantity‘ y decrementa ‘stock_quantity‘.
4. Si alguna prenda no tiene existencias, la transaccion hace ‘ROLLBACK‘ y el cliente recibe un error HTTP 400 sin crear pedidos huerfa

G. Integracion de Pasarelas de Pago

G.1 PixelPay Honduras (Tarjetas de Credito y Debito)
* **Archivos:** ‘/src/services/paymentService.ts‘ (L. 165-275), ‘/src/lib/cardValidation.ts‘, ‘/server.ts‘ (L. 739-781).
* **Moneda de liquidacion:** Lempiras (HNL).
* **Modo Sandbox / Pruebas:** Si no se detectan las variables ‘PIXELPAY_KEY‘ y ‘PIXELPAY_SECRET‘, el sistema activa el simulador bancario interno que responde con los codigos reglamentarios de la CNBS:
* Tarjeta ‘4242 4242 4242 4242‘ **Aprobada** (‘00‘).
* Tarjeta ‘4000 0000 0000 0002‘ **Declinada: Fondos Insuficientes** (‘51‘).
* Tarjeta ‘4000 0000 0000 0004‘ **Declinada: Tarjeta Vencida** (‘54‘).
* Tarjeta ‘4000 0000 0000 0005‘ **Declinada: Fallo de Autenticacion 3D-Secure** (‘3DS_FAILED‘).
* Tarjeta ‘4000 0000 0000 9999‘ **Fallo de Red / Timeout Bancario** (‘96‘).
* **Webhook:** ‘/api/webhooks/pixelpay‘ (L. 856 en ‘server.ts‘).

G.2 PayPal REST API v2 (Pagos Internacionales)
* **Archivos:** ‘/src/services/paymentService.ts‘ (L. 280-380), ‘/server.ts‘ (L. 786-818).
* **Moneda:** Dolares Estadounidenses (USD). Utiliza la tasa oficial de conversion configurada en ‘HNL_TO_USD_RATE = 25.50‘.
* **Endpoints:**
* ‘POST /api/payments/paypal/create-order‘
* ‘POST /api/payments/paypal/capture-order‘
* **Webhook:** ‘/api/webhooks/paypal‘ (L. 883 en ‘server.ts‘).

H. Panel Administrativo de la Tienda

Archivo Frontend: ‘/src/components/AccountModal.tsx‘
Endpoints Backend: ‘/server.ts‘ (Protegidos con ‘requireAuth‘ y ‘requireAdmin‘)
* ‘GET /api/admin/orders‘ (L. 1015): Lista los ultimos 50 pedidos con sus articulos y estado de envio.
* ‘PATCH /api/admin/orders/:id/status‘ (L. 1047): Permite al administrador cambiar el estado a: ‘confirmed‘, ‘processing‘, ‘shipped‘, ‘delivered‘ o ‘cancelled‘.
* ‘GET /api/admin/inventory‘ (L. 1096): Lista todas las variantes de pantalones registradas en la base de datos con su SKU y stock actual.
* ‘PATCH /api/inventory/:variantId‘ (L. 1126): Actualiza el numero de unidades en stock de una variante especifica.
* ‘POST /api/admin/images/upload‘ (L. 1145): Recibe imagenes en base64 (JPEG, PNG, WEBP de hasta 5MB), las valida y las guarda en ‘/public/uploads/‘, retornando la URL publica para asociarla al producto.

I. Sistema Logistico y Envios en Honduras

Archivos: ‘/src/lib/shippingRates.ts‘ y ‘/src/data/hondurasLocations.ts‘

* **Departamentos y Municipios:** ‘HONDURAS_DEPARTMENTS‘ contiene los 18 departamentos y los principales municipios, incluyendo *"La Masica (Sede KAS Store)"* como primera opcion en Atlantida.
* **Tarifario por Zonas Geograficas:**
1. *Zona Atlantida (Local):* L 80.00 (Entrega en 24 hrs por mensajeria local).
2. *Zona Norte (Cortes, Yoro):* L 110.00 (24 a 48 hrs por Cargo Expreso / Mirna).
3. *Zona Centro-Sur (Francisco Morazan, Comayagua, La Paz, Choluteca, Valle):* L 130.00 (48 hrs).
4. *Zona Foranea (Colon, Copan, Olancho, Islas de la Bahia, etc.):* L 150.00 (48 a 72 hrs).
* **Umbral de Envio Gratuito:** Definido en la constante ‘FREE_SHIPPING_THRESHOLD_HNL = 1500‘. En compras de L 1,500.00 o mas, el costo de envio es L 0.00 automaticamente.

J. Canales de Contacto y WhatsApp

* **Boton Flotante:** ‘/src/components/WhatsAppButton.tsx‘ (L. 38). Numero configurado: ‘+504 3305-6175‘. Mensaje predefinido codificado en URL: ‘Hola KAS Store, quisiera informacion sobre los pantalones de mezclilla‘.
* **Cabecera y Barra Superior:** ‘/src/components/Header.tsx‘ (L. 52-58).
* **Pie de Pagina:** ‘/src/components/Footer.tsx‘ (L. 73-81 y L. 208-242).
* **Centro de Contacto:** ‘/src/pages/PoliciesPage.tsx‘ (Pestana ‘contacto‘).
* **Datos de la Sede:** Barrio El Centro, La Masica, Atlantida, Honduras. Correo: ‘k-andradesantos@gmail.com‘.

K. SEO, Metadatos y Paginas Legales

* **Metaetiquetas y Schema.org:** ‘/index.html‘. Contiene el objeto JSON-LD ‘@type: "ClothingStore"‘ con direccion postal en La Masica, telefono, coordenadas geograficas y catalogo indexable.
* **Directivas para Motores de Busqueda:** ‘/public/robots.txt‘.
* **Mapa de URLs:** ‘/public/sitemap.xml‘.
* **Centro de Politicas:** ‘/src/pages/PoliciesPage.tsx‘ con navegacion por pestanas:
* ‘envios‘: Politicas de despacho y transportistas en Honduras.
* ‘devoluciones‘: 10 dias para cambio de talla sin uso.
* ‘terminos‘: Terminos de compra conforme a la Ley de Comercio Electronico (Decreto 149-2014).
* ‘privacidad‘: Manejo etico de datos del comprador.
* ‘contacto‘: Canales oficiales de soporte.
* ‘faq‘: Preguntas frecuentes de clientes.
* **Pagina de Error 404:** ‘/src/pages/NotFoundPage.tsx‘.

L. Notificaciones y Correos Electronicos

* **Estado Actual:** El backend actual registra todos los eventos de confirmacion en la tabla ‘payment_events‘. Las variables para proveedor de correo transaccional (‘RESEND_API_KEY‘, ‘EMAIL_FROM‘, ‘ADMIN_ALERT_EMAIL‘) estan contempladas en ‘.env.example‘ y documentadas en ‘DESPLIEGUE_Y_OPERACIONES.md‘.
* **Notificacion por WhatsApp:** En la arquitectura actual, la notificacion y coordinacion de guias de entrega se realiza de forma directa e inmediata a traves de la API oficial de enlace de WhatsApp (‘https://wa.me/50433056175‘).

---

4. TABLA MAESTRA DE MODIFICACIONES

Esta tabla es la guia de referencia rapida para que el propietario o desarrollador sepa con exactitud que archivo tocar segun la necesidad del negocio:

```
# | Cambio que deseo realizar | Archivos a revisar | Funcion del archivo | Tipo de mo
--- | --- | --- | --- | --- | --- | ---
1 | **Cambiar el Logotipo** | ‘/public/assets/aistudio/Logotipo_KAS_Store_9c74b5ef.pn
2 | **Cambiar colores de la marca** | ‘/src/index.css‘ | Define tokens ‘@theme‘ de Ta
3 | **Modificar textos del banner principal** | ‘/src/pages/HomePage.tsx‘ | Renderiza
4 | **Cambiar los productos destacados** | Panel Admin o ‘/src/data/mockProducts.ts‘
5 | **Anadir un nuevo pantalon** | ‘/src/data/mockProducts.ts‘ y tabla ‘products‘ de
6 | **Cambiar precio de un jean** | Base de datos (‘products.basePriceHnl‘) o ‘/src/d
7 | **Modificar las tallas disponibles** | ‘/src/data/mockProducts.ts‘ y tabla ‘produ
8 | **Subir nuevas fotografias de jeans** | Panel Admin (‘AccountModal.tsx‘ > Subir I
9 | **Cambiar el numero de WhatsApp** | ‘/src/components/WhatsAppButton.tsx‘, ‘Header
10 | **Modificar tarifas de envio por departamento** | ‘/src/lib/shippingRates.ts‘ |
11 | **Cambiar el umbral de envio gratis** | ‘/src/lib/shippingRates.ts‘ | Constante
```

```
12 | **Modificar campos del formulario de compra** | ‘/src/pages/CheckoutPage.tsx‘ y
13 | **Ajustar existencias de inventario** | Panel Admin (‘AccountModal.tsx‘ > Invent
14 | **Habilitar credenciales reales de PixelPay** | Archivo ‘.env‘ en el servidor |
15 | **Habilitar credenciales reales de PayPal** | Archivo ‘.env‘ en el servidor | ‘P
16 | **Modificar politicas legales o terminos** | ‘/src/pages/PoliciesPage.tsx‘ | Con
17 | **Crear o modificar un cupon de descuento** | Tabla ‘coupons‘ en PostgreSQL o ‘/
```

---

5. MAPAS DE FLUJO DE DATOS

5.1 Flujo General: Navegador Frontend API Base de Datos

```
graph TD
    User([Cliente en Navegador]) -->|Solicita Catalogo o Producto| ReactApp[Frontend
    ReactApp -->|fetch ’/api/products’| ExpressServer[Servidor Express API]
    ExpressServer -->|Consulta SQL con Drizzle| PostgreSQL[(Base de Datos Cloud SQL)]
    PostgreSQL -->|Retorna registros y stock| ExpressServer
    ExpressServer -->|JSON con productos hidratados| ReactApp
    ReactApp -->|Renderiza interfaz visual| User
```

5.2 Flujo de Checkout con Reserva Transaccional de Inventario

```
sequenceDiagram
    autonumber
    actor Cliente
    participant Checkout as CheckoutPage.tsx
    participant API as server.ts (/api/orders)
    participant PaymentSvc as paymentService.ts
    participant DB as PostgreSQL (Transaccion BEGIN)

    Cliente->>Checkout: Ingresa direccion en Honduras y confirma pedido
    Checkout->>API: POST /api/orders (items, direccion, metodo, idempotencyKey)
    API->>DB: BEGIN TRANSACTION
    API->>PaymentSvc: reserveInventory(items)
    PaymentSvc->>DB: SELECT stock_quantity FROM inventory WHERE variant_id = $1 FOR U
    alt Hay stock suficiente
        DB-->>PaymentSvc: Existencias bloqueadas
        PaymentSvc->>DB: UPDATE inventory SET stock_quantity = stock - qty, reserved
        API->>DB: INSERT INTO orders (status: ’pending’) RETURNING order_number
        API->>DB: COMMIT TRANSACTION
        API-->>Checkout: 201 Created (Order Number: KAS-2026-XXXX)
        Checkout-->>Cliente: Despliega pasarela de pago seleccionada (Tarjeta / PayPa
    else Stock agotado o insuficiente
        DB-->>PaymentSvc: Existencias menores a cantidad solicitada
        PaymentSvc-->>API: Error: Inventario insuficiente
        API->>DB: ROLLBACK TRANSACTION
        API-->>Checkout: 400 Bad Request
        Checkout-->>Cliente: Alerta: "Prenda agotada. Por favor actualiza tu bolsa."
    end
```

5.3 Flujo de Pago con Tarjeta (PixelPay Honduras)

```
sequenceDiagram
    autonumber
    actor Cliente
    participant Checkout as CheckoutPage.tsx
    participant API as server.ts (/api/payments/card/process)
    participant Gateway as PixelPay API / Simulador CNBS
    participant DB as PostgreSQL
```

```
    Cliente->>Checkout: Digita tarjeta, mes, ano, CVV y hace clic en "Pagar"
    Checkout->>API: POST /api/payments/card/process (orderId, datos tarjeta tokenizab
    API->>Gateway: Enviar solicitud de cobro con importe en HNL
    alt Transaccion Aprobada (Codigo 00)
        Gateway-->>API: Respuesta de aprobacion con authorization_code
        API->>DB: UPDATE orders SET status = 'confirmed', payment_status = 'completed
        API->>DB: confirmInventory(orderId) -> Reduce reserved_quantity de forma defi
        API->>DB: INSERT INTO payments (status: 'captured', card_brand, card_last_fou
        API-->>Checkout: 200 OK (Pago confirmado con exito)
        Checkout-->>Cliente: Pantalla de Pedido Confirmado con numero de guia
    else Transaccion Declinada (Fondos, vencida, etc.)
        Gateway-->>API: Declinada con codigo de rechazo
        API->>DB: INSERT INTO payment_events (event: 'PAYMENT_DECLINED')
        API-->>Checkout: 402 Payment Required (Mensaje amigable segun codigo bancario
        Checkout-->>Cliente: Muestra razon del banco y permite reintentar sin perder
    end
```

---

6. INVENTARIO DE VARIABLES DE ENTORNO

A continuacion se detalla la matriz de variables de entorno del sistema. **Por razones de seguridad, nunca se deben colocar valores reales en archivos bajo control de versiones:**

```
Variable | Servicio | Proposito | Publica o Secreta? | Donde se configura | Obligat
--- | --- | --- | --- | --- | --- | ---
'PORT' | Servidor Express | Puerto de escucha HTTP | Publica (Sistema) | '.env' / Clo
'NODE_ENV' | Entorno | 'development' o 'production' | Publica | '.env' / Dockerfile |
'DATABASE_URL' | PostgreSQL | Cadena de conexion para Drizzle Kit | **Secreta** | '.e
'PGHOST' | Driver pg | Servidor de base de datos | Secreta | '.env' | Si | 'GET /api/
'PGPORT' | Driver pg | Puerto de PostgreSQL (5432) | Publica | '.env' | Si | Verifica
'PGUSER' | Driver pg | Usuario de base de datos | Secreta | '.env' | Si | Verificada
'PGPASSWORD' | Driver pg | Contrasena de base de datos | **Secreta** | '.env' | Si |
'PGDATABASE' | Driver pg | Nombre de base de datos | Publica | '.env' | Si | Verifica
'PGSSLMODE' | Driver pg | Modo SSL ('require' en nube) | Publica | '.env' | Si en nub
'PIXELPAY_KEY' | PixelPay | Llave publica de comercio | Publica de integracion | '.en
'PIXELPAY_SECRET' | PixelPay | Llave secreta para firmar cobros | **Altamente Secreta
'PIXELPAY_AUTH_TOKEN' | PixelPay | Token Bearer para API v2 | **Secreta** | '.env' |
'PAYPAL_API_BASE' | PayPal | URL base (sandbox o live) | Publica | '.env' | Si ('http
'PAYPAL_CLIENT_ID' | PayPal | Identificador de aplicacion | Publica de integracion |
'PAYPAL_CLIENT_SECRET' | PayPal | Clave secreta OAuth de PayPal | **Altamente Secreta
'FIREBASE_PROJECT_ID' | Firebase | Proyecto de autenticacion | Publica | 'firebase-ap
```

---

7. BASE DE DATOS RELACIONAL Y MODELADO DE DATOS

El proyecto utiliza **PostgreSQL** administrado mediante **Drizzle ORM** (definido en '/src/db/schema.ts').

7.1 Tablas y Entidades del Sistema

1. **'users':** Almacena el perfil del cliente sincronizado con Firebase Auth ('uid', 'email', 'full_name', 'phone', 'role': ''admin'' o ''customer
2. **'categories':** Categorias de mezclilla ('slug', 'name', 'gender_target', 'image_url', 'display_order').
3. **'products':** Registro maestro de jeans ('slug', 'name', 'base_price_hnl', 'gender', 'fit_type', 'denim_wash', 'fabric_composition', 'is_pu
4. **'product_images':** Galeria de imagenes de cada prenda con orden de visualizacion ('product_id', 'image_url', 'alt_text', 'is_primary')
5. **'product_variants':** Variantes por combinacion de talla y color ('product_id', 'size', 'color_name', 'color_hex', 'sku', 'price_adjustment
6. **'inventory':** Existencias en tiempo real ('variant_id', 'stock_quantity', 'reserved_quantity', 'low_stock_threshold').

7. **‘coupons‘:** Cupones de descuento promocionales (‘code‘, ‘discount_percent‘, ‘discount_amount_hnl‘, ‘min_purchase_hnl‘, ‘max_use
8. **‘orders‘:** Pedidos realizados (‘order_number‘, ‘user_id‘, ‘guest_email‘, ‘status‘, ‘subtotal_hnl‘, ‘discount_hnl‘, ‘shipping_fee_hnl‘, ‘tota
9. **‘order_items‘:** Snapshot historico de cada articulo comprado (‘order_id‘, ‘product_id‘, ‘product_name_snapshot‘, ‘size_snapshot‘, ‘co
10. **‘payments‘:** Transacciones financieras (‘order_id‘, ‘amount_hnl‘, ‘currency‘, ‘provider‘, ‘status‘, ‘transaction_reference‘, ‘card_branc
11. **‘payment_events‘:** Registro de auditoria y webhooks recibidos (‘order_id‘, ‘provider‘, ‘event_type‘, ‘payload‘).
12. **‘shipments‘:** Informacion de despacho (‘order_id‘, ‘tracking_number‘, ‘carrier_name‘, ‘status‘, ‘estimated_delivery_at‘).
13. **‘wishlist_items‘:** Articulos favoritos guardados por clientes registrados (‘user_id‘, ‘product_id‘).

7.2 Como anadir un nuevo campo en la base de datos

Si en el futuro se necesita anadir un campo (por ejemplo, ‘instagram_url‘ en la tabla de tiendas o ‘delivery_instructions‘ en pedidos):
1. Abrir ‘/src/db/schema.ts‘ y anadir la columna en la definicion de la tabla con su tipo de dato.
2. Ejecutar en terminal: ‘npx drizzle-kit push‘ para aplicar la modificacion a PostgreSQL de forma segura.
3. Actualizar la interfaz TypeScript correspondiente en ‘/src/types.ts‘.

---

8. PROTOCOLO DE SEGURIDAD PARA MODIFICAR EL CODIGO SIN ROMPERLO

Para realizar cualquier modificacion tecnica sin afectar las ventas ni la estabilidad de la tienda, siga rigurosamente este protocolo de 10 pasos:

```
[1. Localizar Archivo]  --> [2. Revisar Dependencias] --> [3. Crear Copia/Rama]
         |
[4. Cambio Minimo]      --> [5. Compilar (npm run build)] --> [6. Validar Linter (npm
         |
[7. Probar Caso Feliz]  --> [8. Probar Errores/Bordes]  --> [9. Revisar Logs de Conso
         |
[10. Aprobacion y Despliegue a Produccion]
```

Reglas Criticas

1. **Nunca modifique archivos de pago sin validar en Sandbox:** Toda prueba de tarjetas debe realizarse primero con las tarjetas de pru
2. **Respeto a las transacciones de inventario:** Cualquier cambio en ‘/api/orders‘ debe mantener el bloque ‘BEGIN ... COMMIT / ROLLE
3. **Validacion de tipos:** Ejecutar ‘npm run lint‘ (‘tsc --noEmit‘) antes de cualquier despliegue. Si el compilador detecta un error de tipos,

---

9. GLOSARIO Y GUIA PRACTICA PARA EL PROPIETARIO

Para facilitar la comunicacion entre el propietario de la tienda y su equipo tecnico, a continuacion se definen los terminos fundamentales en lenguaje accesible:

* **Frontend:** La parte visual de la tienda que el cliente ve en su celular o computadora (botones, fotos, textos y carrito).
* **Backend (API):** El servidor que opera tras bambalinas. Es el encargado de calcular los precios reales, descontar el inventario y hablar con los bancos.
* **Base de Datos (PostgreSQL):** El archivo digital permanente donde se guardan de forma organizada los pedidos, los clientes y la lista de pantalones.
* **Transaccion de Base de Datos:** Una regla que asegura que todas las operaciones de una compra se completen con exito o no se aplique ninguna. Evita que a un cliente se le cobre si el sistema se desconecta antes de guardar el pedido.
* **Idempotencia (‘idempotency_key‘):** Una proteccion que evita que un cliente sea cobrado dos veces si presiona repetidamente el boton de pago por impaciencia o conexion lenta.
* **Webhook:** Una notificacion automatica que la pasarela de pago (PixelPay o PayPal) envia a nuestro servidor para avisarle que un cobro fue autorizado.
* **Tokenizacion:** El metodo por el cual los datos de una tarjeta se convierten en un codigo seguro, permitiendo cobrar sin que nadie en KAS Store pueda ver el numero de la

tarjeta.

---

10. ESTADO DE VERIFICACION, LIMITACIONES CONOCIDAS Y OBSERVACIONES

10.1 Modulos Verificados y Funcionales

* [x] Navegacion completa en movil, tableta y escritorio.
* [x] Catalogo con filtros reactivos y sincronizacion de base de datos.
* [x] Detalle de producto con bloqueo de combinaciones agotadas.
* [x] Bolsa de compras y drawer interactivo con persistencia local.
* [x] Checkout adaptado con los 18 departamentos y tarifas de flete de Honduras.
* [x] Reserva transaccional con ‘SELECT FOR UPDATE‘ en PostgreSQL.
* [x] Simulador bancario de pruebas con tarjetas de aprobacion y codigos de rechazo CNBS.
* [x] Integracion de ordenes de PayPal REST API v2.
* [x] Panel administrativo con actualizacion de estados de pedidos, edicion de inventario en vivo y subida de fotos.
* [x] Marcado Schema.org, ‘robots.txt‘, ‘sitemap.xml‘ y pagina 404 personalizada.

10.2 Limitaciones Conocidas y Pasos Pendientes para el Propietario

* **Credenciales Reales en Produccion:** Las pasarelas operan actualmente en modo de prueba seguro (sandbox). Para procesar cobros bancarios reales, el propietario de KAS Store debe firmar el contrato comercial con PixelPay Honduras y configurar las llaves en las variables de entorno.
* **Revision Legal Hondurena:** Los textos de las politicas fueron redactados tomando como modelo el Decreto 149-2014 (Ley de Comercio Electronico) y el Decreto 24-2008 (Proteccion al Consumidor). Se recomienda su validacion final por un profesional del derecho en Honduras.
* **Notificaciones por Correo:** El sistema registra los eventos de compra en la base de datos y provee enlaces a WhatsApp; la emision de correos automaticos requerira configurar las credenciales SMTP o API de Resend en el servidor.

Manual tecnico elaborado por Mora-Grafic’s Studio para KAS Store. La Masica, Atlantida, Honduras. Todos los derechos reservados.